Problem 4: Initial Distribution

The functions `f` and `g` from Homework 1 were updated. Originally, they accepted a single integer `x` representing the initial state. The modified versions now accept a vector `q` representing an initial probability distribution over the state space. This allows for simulations where the starting state is chosen randomly according to `q`.

Pseudocode for `f(P, q, y, n)`

The function `f` now calculates $P(X_n = y)$ given an initial distribution q. It does this by computing the distribution at time n, which is qP^n, and then extracting the probability of being in state y.

procedure F(P, q, y, n)
  Validate inputs: $n \geq 0$, P is square, y is a valid state, q is a valid probability vector.
  $A \leftarrow$ numpy array of P
  $q_{\text{vec}} \leftarrow$ numpy array of q
  $A_n \leftarrow$ matrix_power(A, n)
  $\text{dist}_n \leftarrow q_{\text{vec}} \cdot A_n$
  return $\text{dist}_n[y-1]$

Pseudocode for `g(P, q, n, seed)`

The function `g` simulates a path of the Markov chain. The initial state X_0 is now drawn from the distribution q. The rest of the path is generated sequentially based on the transition matrix P.

procedure G(P, q, n, seed)
  if $n = 0$ **then return** []
  rng $\leftarrow$ RNG with optional seed
  $N \leftarrow$ number of states
  states $\leftarrow \{0, 1, \ldots, N-1\}$
  initial_state $\leftarrow$ rng.choice(states, $p = q$)
  current $\leftarrow$ initial_state
  path $\leftarrow$ []
  for $i = 1$ to n **do**
    probs $\leftarrow P[\text{current}]$
    next_state $\leftarrow$ rng.choice(states, $p =$ probs)
    path.append(next_state + 1)
    current $\leftarrow$ next_state
  return path

Code

The following is the Python implementation of the modified functions.

```python
import numpy as np
from typing import Sequence, Union

def f(P: Union[Sequence[Sequence[float]], np.ndarray], q: Union[Sequence[float],
    np.ndarray], y: int, n: int) -> float:
    """
    Return P(X_n = y) for a discrete-time Markov chain given an initial
        distribution q.

    Parameters
    - P: N x N transition matrix (rows sum to 1). Accepts nested lists or numpy
        arrays.
    - q: initial probability distribution vector (0-indexed).
```

```
    - y: state in {1, ..., N} (1-based indexing).
    - n: non-negative integer time step.

    Returns
    - probability as a float
    """
    if n < 0:
        raise ValueError("n must be a non-negative integer")

    A = np.array(P, dtype=float)
    if A.ndim != 2 or A.shape[0] != A.shape[1]:
        raise ValueError("P must be a square (N x N) matrix")

    N = A.shape[0]
    if not (1 <= y <= N):
        raise ValueError("y must be an integer in 1..N (1-based indexing)")

    q_vec = np.array(q, dtype=float)
    if q_vec.ndim != 1 or q_vec.shape[0] != N:
        raise ValueError(f"q must be a vector of length {N}")
    if not np.all(q_vec >= 0) or not np.isclose(np.sum(q_vec), 1.0):
        raise ValueError("q must be a valid probability distribution (non-negative
            , sums to 1)")

    An = np.linalg.matrix_power(A, n)
    dist_n = q_vec @ An
    return float(dist_n[y - 1])


def g(P: Union[Sequence[Sequence[float]], np.ndarray], q: Union[Sequence[float],
    np.ndarray], n: int, seed: int | None = None) -> list:
    """
    Simulate one path (y1, ..., yn) of the Markov chain starting from an initial
        distribution `q`.

    Parameters
    - P: N x N transition matrix (rows sum to 1).
    - q: initial probability distribution vector.
    - n: number of steps to simulate (non-negative integer).
    - seed: optional RNG seed for reproducibility.

    Returns
    - list of length `n` with states in 1..N (1-based) representing X1..Xn.
    """
    if n < 0:
        raise ValueError("n must be a non-negative integer")

    A = np.array(P, dtype=float)
    if A.ndim != 2 or A.shape[0] != A.shape[1]:
        raise ValueError("P must be a square (N x N) matrix")

    N = A.shape[0]
    q_vec = np.array(q, dtype=float)
    if q_vec.ndim != 1 or q_vec.shape[0] != N:
        raise ValueError(f"q must be a vector of length {N}")
    if not np.all(q_vec >= 0) or not np.isclose(np.sum(q_vec), 1.0):
        raise ValueError("q must be a valid probability distribution (non-negative
            , sums to 1)")
```

```python
    if n == 0:
        return []

    rng = np.random.default_rng(seed)

    states = np.arange(N)
    initial_state_0_indexed = rng.choice(states, p=q_vec)

    path = []
    current = initial_state_0_indexed
    for _ in range(n):
        probs = A[current]
        next_state = rng.choice(N, p=probs)
        path.append(int(next_state + 1))
        current = next_state

    return path
```

Listing 1: Updated `f` and `g` functions from `MATH541PS1/HW1/markov.py`

Problem 5: Metropolis-Hastings for Cryptography

This problem involved using the Metropolis-Hastings algorithm to find the decryption key for a substitution cipher. The state space is the set of all 26! permutations of the alphabet. The target distribution is based on the likelihood of the decrypted text, which is calculated using a transition matrix of English letters derived from "War and Peace".

Pseudocode for Metropolis-Hastings

The algorithm starts with a random permutation (key) and iteratively proposes new keys by swapping two letters. A new key is accepted with a probability that depends on how much it improves the likelihood of the decrypted text.

procedure METROPOLIS-HASTINGS(text, M, β, iterations)
  $s \leftarrow$ random permutation of the alphabet
  $\log Pl(s) \leftarrow$ calculate_log_likelihood(s, text, M)
  for $i = 1$ to iterations **do**
    $s' \leftarrow s$ with two random letters swapped
    $\log Pl(s') \leftarrow$ calculate_log_likelihood(s', text, M)
    $\alpha \leftarrow \exp(\beta \cdot (\log Pl(s') - \log Pl(s)))$
    if random_uniform$(0, 1) < \alpha$ **then**
      $s \leftarrow s'$
      $\log Pl(s) \leftarrow \log Pl(s')$
  return s

Code

The following code was used to solve the decryption problem. It implements the Metropolis-Hastings algorithm and uses multiprocessing to run multiple simulations in parallel to find the best decryption key.

```python
import numpy as np
import random
import math
import re
import multiprocessing
```

```python
# The encrypted message
encrypted_message = "col hlyg fms wit fh gtww pommf mtzty poosqet jfc pommf mtzty
    ryc col efvt pommf mtzty col stjtyg fms fyolms ylm pommf mtzty soam col wtg
    pommf mtzty In col pizt pommf mtzty jtt go qwims goo colyt et fjv col ik fms ig
     nwfc pommf atyt fms pfet ght vmoa at om poimp qttm ahfgj vmoa qogh at imjist
    ig jfc go jhe goo colyt qlg frhimp qttm htfygj coly womp jo koy oghty tfrh
    vmoam atzt Imstyjgfms col efvt poggf kttwimp ie hoa col gtww afmmf uljg tet plc
     oghty fmc kyoe ghij ptg aolwsmg col ok ghimvimp ie ahfg roeeigetmgj klww fh
    tct so jo fms ylwtj ght vmoa col wozt go jgyfmptyj mo atyt"

# Create a mapping from character to index (a=0, b=1, ...)
char_to_int = {chr(ord('a') + i): i for i in range(26)}

def get_transition_matrix(filename="VSCode/MATH541PS1/HW2/
    war_and_peace_transition_matrix_letters_only.csv"):
    """
    Reads the transition matrix from a CSV file.
    """
    try:
        # Adding a small epsilon for smoothing to prevent log(0)
        matrix = np.genfromtxt(filename, delimiter=',')
        if np.count_nonzero(matrix) == 0:
             # Fallback if matrix is empty or all zeros
            return np.full((26, 26), 1/26)
        matrix += 1e-12
        matrix /= matrix.sum(axis=1, keepdims=True)
        return matrix
    except OSError:
        print(f"Error: Could not read {filename}.")
        print("Please make sure the file is in the correct directory.")
        # Return a uniform probability matrix if the file is not found
        return np.full((26, 26), 1/26)


def calculate_log_pl(s, text, M):
    """
    Calculates the log of Pl(s) for a given permutation s and text.
    """
    log_pl = 0.0

    # Create a decryption map from the permutation
    decryption_map = {chr(ord('a') + i): s[i] for i in range(26)}

    # Clean the text to treat it as one continuous string of letters
    cleaned_text = re.sub(r'[^a-zA-Z]', '', text).lower()

    for i in range(len(cleaned_text) - 1):
        current_char_encrypted = cleaned_text[i]
        next_char_encrypted = cleaned_text[i+1]

        # Decrypt characters
        current_char_decrypted = decryption_map.get(current_char_encrypted)
        next_char_decrypted = decryption_map.get(next_char_encrypted)

        if current_char_decrypted and next_char_decrypted:
            u = char_to_int[current_char_decrypted]
            v = char_to_int[next_char_decrypted]
            log_pl += math.log(M[u, v])
```

```python
    return log_pl


def metropolis_hastings(text, M, beta=0.1, iterations=100000, run_id=None):
    """
    Performs the Metropolis-Hastings algorithm to find the decryption key.
    """
    # Start with a random permutation
    s = list('abcdefghijklmnopqrstuvwxyz')
    random.shuffle(s)
    s = "".join(s)

    # Calculate the initial log Pl
    log_pl_s = calculate_log_pl(s, text, M)

    for i in range(iterations):
        # Propose a new permutation by swapping two letters
        s_prime_list = list(s)
        i1, i2 = random.sample(range(26), 2)
        s_prime_list[i1], s_prime_list[i2] = s_prime_list[i2], s_prime_list[i1]
        s_prime = "".join(s_prime_list)

        # Calculate the log Pl for the new permutation
        log_pl_s_prime = calculate_log_pl(s_prime, text, M)

        # Calculate the acceptance ratio
        if log_pl_s_prime > log_pl_s:
            acceptance_ratio = 1.0
        # This check is important to prevent math range errors
        elif log_pl_s == -float('inf'):
            acceptance_ratio = 1.0
        else:
            acceptance_ratio = math.exp(beta * (log_pl_s_prime - log_pl_s))

        # Accept or reject the new permutation
        if random.random() < acceptance_ratio:
            s = s_prime
            log_pl_s = log_pl_s_prime

    return s
```

Listing 2: Metropolis-Hastings implementation from `MATH541PS1/HW2/markov.py`